React.js Master Guide

From foundational concepts to advanced patterns, this comprehensive guide will transform you into a confident React developer ready for professional work.

Made with GAMMA

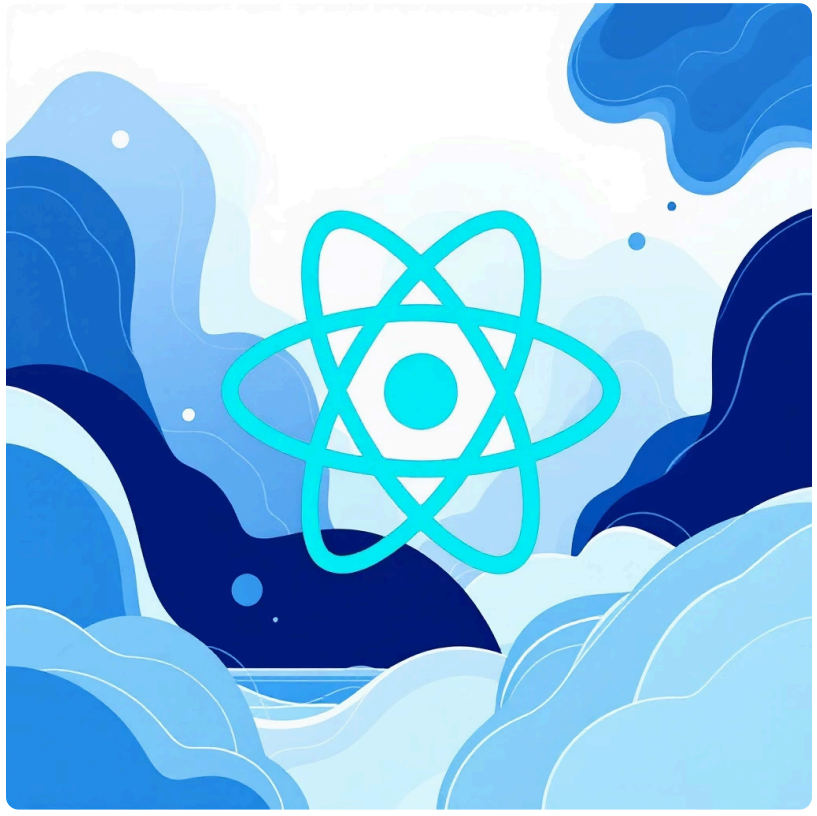
What is React?

React is a JavaScript library for building user interfaces, particularly web applications. It enables developers to create interactive UIs through a component-based architecture.

React revolutionises web development by introducing the Virtual DOM - a JavaScript representation of the actual DOM that enables efficient updates and rendering.

Key Benefits

- Declarative programming model
- Component reusability
- Efficient Virtual DOM updates
- Strong ecosystem and community



Made with GAMMA

Virtual DOM vs Real DOM

Real DOM

Direct manipulation of browser's Document Object Model. Heavy operations cause entire page reflows and repaints.

- Slow updates
- Memory intensive
- Direct browser manipulation

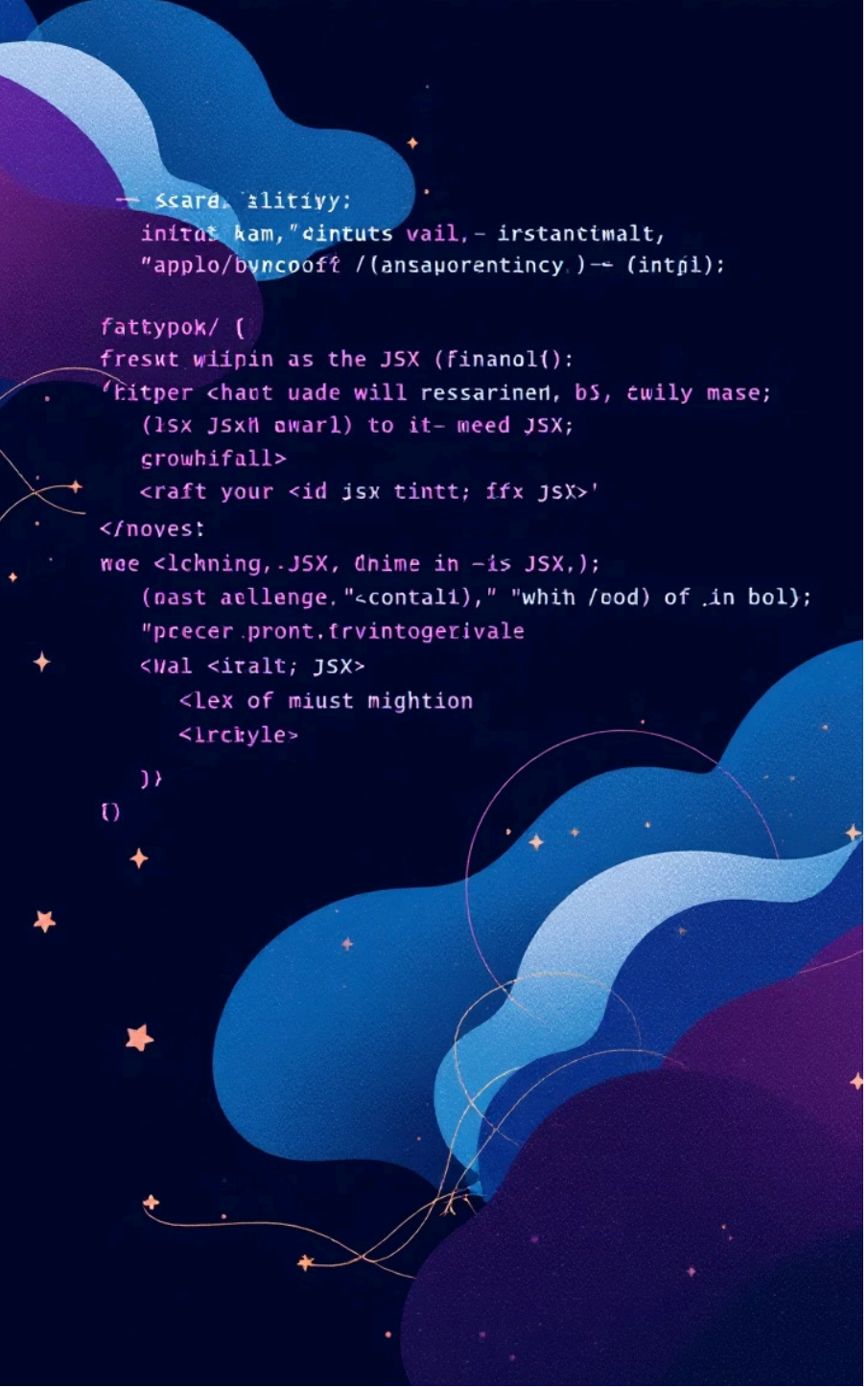
Virtual DOM

Lightweight JavaScript representation. React calculates optimal changes before updating the real DOM efficiently.

- Fast diffing algorithm
- Batch updates
- Memory efficient

This approach makes React applications significantly faster and more responsive than traditional DOM manipulation methods.

Made with GAMMA



JSX: JavaScript + HTML

JSX (JavaScript XML) is React's syntax extension that lets you write HTML-like code within JavaScript. It gets transpiled to regular JavaScript function calls.

JSX Syntax

```
const element = <h1>Hello, World!</h1>;
```

Looks like HTML but it's actually JavaScript

JavaScript Expressions

```
const name = "React";
const element = <h1>Hello, {name}!</h1>
```

Embed variables and expressions using curly braces

Made with GAMMA

Components: Building Blocks

Function Components (Modern)

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

// Arrow function syntax
const Welcome = (props) => {
  return <h1>Hello, {props.name}</h1>;
};
```

Recommended approach for new React applications

Class Components (Legacy)

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Still supported but function components with hooks are preferred

Made with GAMMA

Props, State & Events

Props

Read-only data passed from parent to child components. Props enable component reusability and data flow.

```
function Button({text, onClick}) {
  return <button onClick=
{onClick}>{text}</button>;
}
```

State

Mutable data that components own and manage. State changes trigger re-renders and UI updates.

```
const [count, setCount] =
useState(0);
```

Events

React uses SyntheticEvents - a cross-browser wrapper around native events providing consistent behaviour.

```
const handleClick = (e) => {
 e.preventDefault();
 // Handle event
};
```

Made with GAMMA

Essential React Hooks

01

useState

Manages local component state. Returns current state and setter function for updates.

02

useEffect

Handles side effects like API calls, subscriptions, and DOM manipulation. Replaces lifecycle methods.

03

useContext

Consumes React Context without nesting. Provides access to context values throughout component tree.

04

useReducer

Manages complex state logic. Alternative to useState when state transitions are intricate.

Made with GAMMA

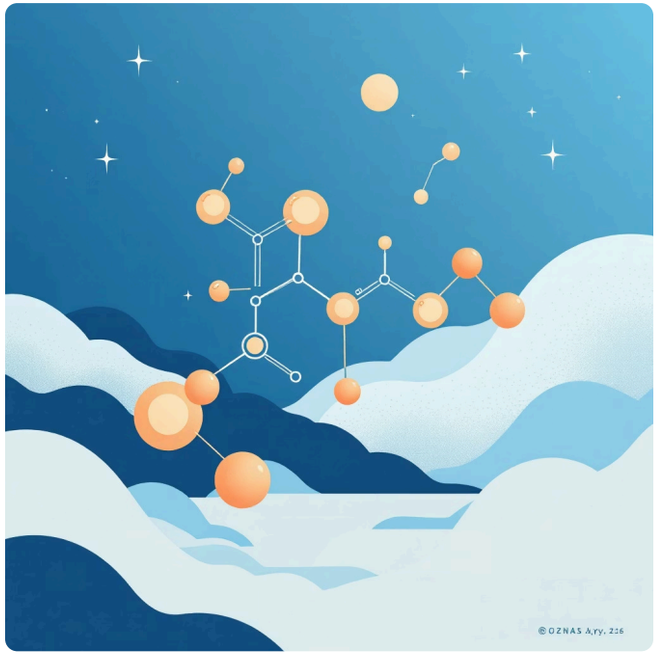
Advanced React Patterns

Render Props

Share code between components using a prop whose value is a function. Enables flexible component composition.

Higher-Order Components

Functions that take a component and return a new enhanced component. Useful for cross-cutting concerns.



Compound Components

Multiple components that work together to form a complete UI. Provides flexible and intuitive APIs.

These patterns help create reusable, maintainable components that scale with your application's complexity.

Made with GAMMA

Performance Optimization

React.memo

Prevents unnecessary re-renders by memoizing component output. Only re-renders when props actually change.

useMemo & useCallback

Cache expensive calculations and function references. Essential for optimizing complex computations and event handlers.

Code Splitting

Break your app into smaller chunks loaded on demand. Use React.lazy and Suspense for dynamic imports.

Made with GAMMA

React Developer Roadmap

Master the Fundamentals

Components, Props, State, JSX, and Essential Hooks form your React foundation. Practice building interactive UIs.

Learn React Router

Add navigation and routing to create multi-page applications. Master nested routes and dynamic parameters.

State Management

Explore Context API, Redux Toolkit, and modern alternatives like Zustand for complex application state.

Testing & Production

Learn testing with Jest and React Testing Library. Deploy with Next.js or modern frameworks for production-ready apps.

Your journey to becoming a professional React developer starts with consistent practice and building real projects. Happy coding!

Made with GAMMA